ZERO SHIELD · SECURITY ASSESSMENT

payments-api

Static Application Security Testing

zeroshield.ai
SAST Report

PROJECT	SCAN TYPE	REPORT GENERATED	PLATFORM
payments-api	SAST	Jul 23, 2026 · 05:32 PM	zeroshield.ai

Risk score
F
92
Critical risk

Total Issues
5
1 C · 2 H · 1 M · 1 L

Critical
1
Immediate remediation

High
2
Current sprint priority

Scan Time
2m 14s
Completed successfully

Vulnerability breakdown



■ Critical **1** ■ High **2** ■ Medium **1** ■ Low **1**

Static analysis findings for this scan. KPI summary first; catalog and per-finding detail follow.

How to read: Scan the catalog for priority items, then open matching detail cards for location, evidence, and remediation guidance.

Platform: Zero Shield · **zeroshield.ai**

SEVERITY GUIDANCE

Critical: Immediate remediation required — severe exploitability / impact.

High: Remediate in the current cycle — significant risk if left open.

Medium: Schedule remediation — real risk with moderate urgency.

Low: Track and harden opportunistically — limited immediate impact.

Priority action: 1 Critical · 2 High — remediate these items first. Full findings catalog and evidence follow. More at zeroshield.ai.

Findings Catalog

5 issues

#	SEVERITY	FINDING	LOCATION
1	CRITICAL	SQL injection via string-concatenated query	`services/query_builder.py:87`
2	HIGH	Exposed database URL written to stdout	`backup.py:42`
3	HIGH	Plaintext passwords copied into backup artifacts	`backup.py:118`
4	MEDIUM	Hard-coded restore path reduces environment isolation	`restore.py:23`
5	LOW	Verbose exception details returned to clients	`api/errors.py:55`

Detailed Findings

Each card includes location, description, evidence, and remediation. Technical values use monospace.

CRITICAL

#1 SQL injection via string-concatenated query

Severity	Critical
Where it was found	`services/query_builder.py:87`
Application area	payments-api

DESCRIPTION

User-controlled input is concatenated into a SQL statement, allowing an attacker to alter the query logic and exfiltrate or modify data.

EVIDENCE

```
cursor.execute("SELECT * FROM users WHERE id=" + user_id)
```

REMEDIATION

Use parameterized queries / bound parameters. Never concatenate user input into SQL.

MAPPINGS

OWASP Top 10	`Injection`

HIGH

#2 Exposed database URL written to stdout

Severity	High
Where it was found	`backup.py:42`
Application area	payments-api

DESCRIPTION

Database connection details are printed to standard output and may land in logs accessible to unintended parties.

EVIDENCE

```
print(db_url)
```

REMEDIATION

Remove debug logging of secrets. Use structured logging with redaction.

MAPPINGS

OWASP Top 10	`Security Logging and Monitoring Failures`

HIGH

#3 Plaintext passwords copied into backup artifacts

Severity	High
Where it was found	`backup.py:118`
Application area	payments-api

DESCRIPTION

User password fields are persisted into backup payloads in cleartext.

EVIDENCE

```
password=user.password,
```

REMEDIATION

Exclude credential fields from backups. Store only hashed secrets.

MEDIUM

#4 Hard-coded restore path reduces environment isolation

Severity	Medium
Where it was found	`restore.py:23`
Application area	payments-api

DESCRIPTION

A fixed backup path is embedded in source, which can cause incorrect restores across environments.

EVIDENCE

```
sqlite_file_name = "backups/26-11-2025-Backup.db"
```

REMEDIATION

Load restore paths from configuration or environment variables.

LOW

#5 Verbose exception details returned to clients

Severity	Low
Where it was found	`api/errors.py:55`
Application area	payments-api

DESCRIPTION

Raw exception strings are returned in API responses and may leak internals.

EVIDENCE

```
return JSONResponse({"detail": str(exc)}, status_code=500)
```

REMEDIATION

Return generic client errors; log full details server-side only.